Tutorial 05: El Escudo Arquitectónico

Nginx, Gunicorn y las bases del Strangler Pattern en AWS

Prof. Nicolás Ramírez Vélez - Arquitectura de Software 2026

1. Objetivo del Laboratorio

En este tutorial abandonaremos el servidor de desarrollo de Django (`runserver`) y construiremos una arquitectura de producción de tres capas. Utilizaremos **Nginx** como Proxy Inverso y **Gunicorn** como servidor WSGI. Esta configuración es el requisito fundamental para implementar el *Strangler Pattern* en el futuro.

Prerrequisitos:

- Tutorial 04 completado (Aplicación dockerizada).
- Instancia EC2 de AWS Academy operativa.

2. Paso 1: Preparar a Django para Producción (Gunicorn)

El comando `runserver` no es seguro ni eficiente para manejar tráfico real. Usaremos Gunicorn, un servidor profesional para Python.

2.1. 1.1 Actualizar dependencias

Añada `gunicorn` a su archivo `requirements.txt`:

```
Django>=4.2,<5.0
djangorestframework
psycopg2-binary
gunicorn  # <-- NUEVO SERVIDOR WSGI
```

3. Paso 2: El Cerebro del Tráfico (nginx.conf)

Vamos a crear el archivo de configuración que le dirá a Nginx cómo enrutar las peticiones. Cree una carpeta llamada `nginx` en la raíz de su proyecto y dentro cree un archivo llamado `nginx.conf`.

```
# nginx/nginx.conf

# 1. Definimos nuestro sistema backend (Django + Gunicorn)
# El nombre 'web' debe coincidir con el nombre del servicio en docker-
    compose
upstream django_backend {
    server web:8000;
```

```
}

server {
    # Nginx escuchara en el puerto web estandar (80)
    listen 80;

    # Ignorar logs de favicon para no saturar el servidor
    location = /favicon.ico { access_log off; log_not_found off; }

    # 2. Proxy Inverso: Todo el trafico raiz va al backend de Django
    location / {
        proxy_pass http://django_backend;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_redirect off;
    }

    # Nota Arquitectonica: En el futuro, aqui agregaremos una nueva
    # regla 'location /api/v2/' que apuntara a nuestro microservicio
    Flask
    # para ejecutar el Strangler Pattern.
}
```

4. Paso 3: Orquestación Total (docker-compose.yml)

Ahora debemos modificar nuestro `docker-compose.yml` para incluir a Nginx, aislar a Django del exterior y arrancar con Gunicorn.

Cambios críticos:

1. El servicio `web` ya NO expone puertos al host (borramos `ports: - "8000:8000"`). Solo Nginx podrá hablar con él en la red interna de Docker.
2. Cambiamos el `command` del servicio `web` para usar Gunicorn.
3. Añadimos el servicio `nginx` exponiendo el puerto 80.

Reemplace el contenido de su `docker-compose.yml` con lo siguiente:

```
version: '3.8'

services:
  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=tienda_db
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=secret_password

  web:
    build: .
    # CAMBIO 1: Usamos Gunicorn en lugar de runserver
    # 'tienda_project' debe ser el nombre de la carpeta donde esta su
    wsgi.py
    command: gunicorn tienda_project.wsgi:application --bind
    0.0.0.0:8000
```

```
    volumes:
      - .:/app
    depends_on:
      - db
    environment:
      - DB_NAME=tienda_db
      - DB_USER=postgres
      - DB_PASSWORD=secret_password
      - DB_HOST=db
      - DB_PORT=5432
    # NOTA: Ya no hay seccion 'ports' aqui. Django esta aislado.

  # CAMBIO 2: Agregamos el servicio Nginx
  nginx:
    image: nginx:1.25-alpine
    ports:
      - "80:80"  # Nginx es el unico que da la cara a internet
    volumes:
      # Inyectamos nuestro archivo de configuracion en el contenedor
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - web

volumes:
  postgres_data:
```

5. Paso 4: Despliegue en AWS EC2

Suba sus cambios a GitHub (commit y push). Luego, ingrese por SSH a su instancia EC2 en AWS Academy.

```
# 1. Entre a la carpeta de su proyecto en EC2
cd mi-proyecto-django

# 2. Descargue los nuevos cambios (nginx.conf y el nuevo docker-compose)
git pull origin main

# 3. Reconstruya y levante la nueva arquitectura
docker compose up -d --build

# 4. Verifique que los TRES contenedores esten corriendo (db, web, nginx)
docker ps
```

5.1. 4.1 Configurar el Security Group (Firewall)

Como Nginx ahora escucha en el puerto web estándar (HTTP), debemos abrirlo en AWS:

1. En la consola de AWS, vaya a **EC2** → **Security Groups**.
2. Seleccione el grupo asociado a su instancia.
3. **Edit inbound rules** → Add rule.

4. **Type:** HTTP → **Port range:** 80 → **Source:** 0.0.0.0/0.

5. Save rules.

6. Entrega de Resultados

Para validar la correcta implementación arquitectónica, debe entregar las siguientes evidencias en la plataforma:

1. **Evidencia de Enrutamiento (Browser):** Captura de pantalla de su navegador consumiendo la API de compras en `http://<IP-EC2>/api/v1/comprar/`. **IMPORTANTE:** La URL *no debe tener el puerto 8000*. Si funciona sin el puerto, significa que Nginx está enrutando correctamente por el puerto 80.

2. **Evidencia de Aislamiento (Terminal Local):** Intente acceder a `http://<IP-EC2>:8000`. Debe fallar (timeout). Tome una captura. Esto demuestra que Django está protegido y aislado de internet, accesible solo a través del Proxy Inverso.

3. **Evidencia de Orquestación (Terminal AWS):** Captura de pantalla ejecutando el comando `docker ps` en la consola de AWS donde se vean claramente los contenedores `nginx`, `web` y `db` en estado Üp".